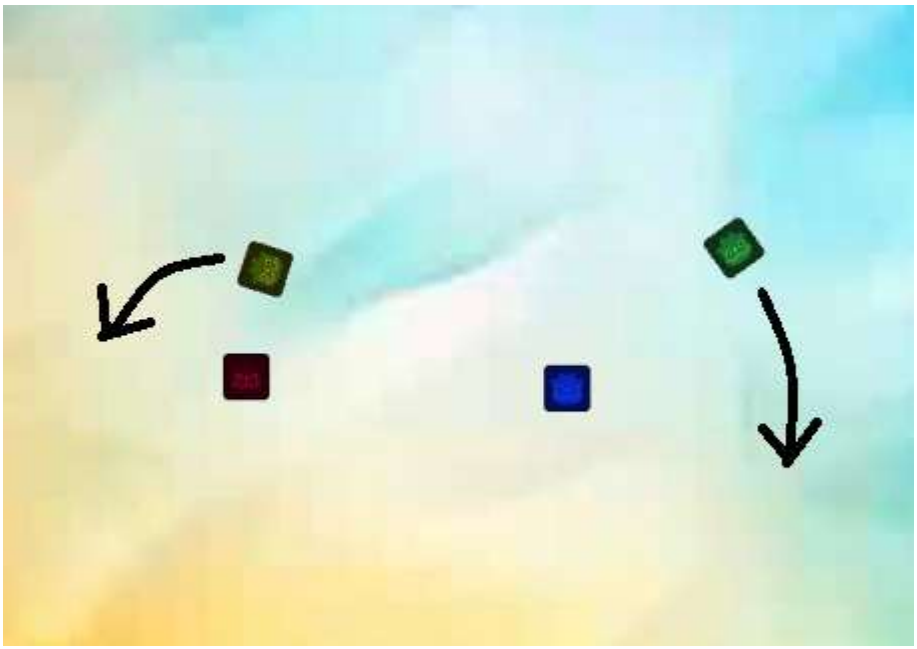
Practice 1: Rotations and movement. Simple math and programming exercises

Let's start with a new challenge... With your actual knowledge, I want to develop something like this image.



The main idea here is to have two icons of 32x32 pixels. One of them, the red is assigned with the left mouse button, and the blue one is assigned with the right mouse button.

Orbiting them, we will have two other icons with other colors. The main challenge here is that you cannot use the usual attributes to change the position of these orbiting icons. You can only modify the entire transformation matrix of them. In other words, you can only assign values to <<Sprite2D>>.transform.

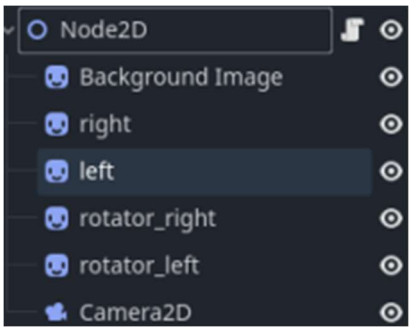
To summarize, these are the rules of this practice:

- Right mouse button will set the position of the 32x32 blue icon.
- Left mouse button will set the position of the red icon.
- All positions must be independent from the viewport coordinates.
- Center of the camera must be in the same position as the red icon.
- Students must create two orbiting icons to both previously defined.
- The orbiting icons must have these exported attributes (that can be seen in the inspector) to set their configuration:
    - Distance: It is the distance to its origin (the other icon defined with the mouse).

    - Speed of rotation: The user can define the speed of both orbiting icons (separately).
- The most important rule is that the orbiting icons must rotate using transformation matrices. So, you cannot use .rotate, global_position or .position on them. **You must modify the values only inside the transform**.
- I want you to detect when both orbiting icons are closer than 32 pixels using the distance_to function their current global_position (here you can use that variable to read the world position). -write something in the console-

Some instructions and constraints

1.- Follow this scene structure:



You can't use the tree hierarchy to help you solve it. Assing colors and defining all as expected.

2.- Assing a reference for them in the code to ease access to them:

```
@onready var right: Sprite2D = $right
@onready var left: Sprite2D = $left
@onready var r_right: Sprite2D = $rotator_right
@onready var r_left: Sprite2D = $rotator_left
@onready var camera_2d: Camera2D = $Camera2D

@export var dist_r: float = 5.0
@export var dist_l: float = 10.0
@export var speed_r: float = 10.0
@export var speed_l: float = 20.0
```

3.- As in previous exercises, assign the position with the mouse when the user presses the button. Also assign the position of the camera. You will need a function like this one to change the viewport coordinates to the world coordinates:

```
func viewport_post_to_world_coordinates( event_pos: Vector2)-> Vector2:
    return get_viewport_transform().affine_inverse() * event_pos
```

4.- Use the _process function to recalculate the transform of the orbiting cells. As explained before, you cannot assign positions or rotations directly. So, you must create a transformation matrix that rotates the current one of the icons. To do so, I recommend using the one previously studied using the delta parameter (to move with a constant velocity).

In this way, if every frame we multiply the current transform matrix with this new one, we will get the expected result.

The order of multiplications is important, and the distances to the orbiting point should be set with Vector2 (0, distance_r or l).

In this way, if we multiply by this matrix every frame, we will be sending the icon far away. We don't want this. So, it is important to copy the origin of the right or left icons that this new icon will orbit.

5.- If in this point you achieved the orbiting effect of the icons; you can configure a last condition to prove if both orbiting icons are 32 px of distance each other. Use the function distance_to with the position of both icons, because you are not assigning any position here, you can use that attribute here to simplify the code.